Math4AI Capstone Project Report

Suleyman Allahverdiyev Seljan Khasiyeva Jeyhuna Sevdiyeva

Abstract

This capstone investigates one of the central questions in machine learning: when is a simple linear classifier sufficient, and when is a non-linear model necessary? To answer this, we compare two models implemented fully from scratch using NumPy: softmax regression and a one-hidden-layer neural network. The comparison is carried out on three datasets of increasing complexity: a linearly separable Gaussian dataset, a non-linear moons dataset, and a high-dimensional handwritten digits dataset. The evidence includes dataset geometry, decision-boundary comparison, training and validation dynamics, hidden-capacity ablation, optimizer behavior, and uncertainty analysis through confidence histograms, entropy boxplots, and reliability diagrams. The results show that softmax regression is highly effective on linearly separable data and remains a strong baseline on more realistic tasks, but its linear structure fundamentally limits it on non-linear problems. The neural network, by introducing a learned hidden representation, becomes necessary when the class structure is curved or otherwise not linearly separable.

1 Introduction

In modern machine learning, one of the most important scientific and practical questions is determining the right model complexity for a given problem. A simple model is easier to interpret, faster to train, and often more stable, but it may fail if the data has structure that cannot be represented in a simple way. A more complex model can learn richer patterns, yet that extra flexibility is not always necessary and can introduce additional optimization difficulty.

The central question of this project is therefore:

When is a linear model sufficient, and when do we need a non-linear model?

To study this, we compare two classifiers:

- **Softmax Regression**, which is a linear probabilistic classifier
- **One-Hidden-Layer Neural Network**, which introduces a non-linear hidden representation

This comparison is scientifically meaningful because it isolates the effect of representational power. The softmax model can only produce linear decision boundaries in the original feature space. By contrast, the neural network applies a learned transformation before classification, allowing it to reshape the geometry of the data and represent curved or more complex boundaries.

The study uses three datasets of increasing difficulty:

- **Linear Gaussian dataset**: a low-dimensional, linearly separable problem
- **Moons dataset**: a low-dimensional but explicitly non-linear problem
- **Digits dataset**: a high-dimensional, multi-class handwritten digit benchmark

These datasets create a progression from simple linear structure to non-linear synthetic structure and then to realistic high-dimensional classification. This design allows the report not only to compare final performance, but also to explain *why* the models behave differently across settings.

2 Background and Mathematical Setup

We study supervised classification with input data

$$X \in \mathbb{R}^{n \times d}$$

and labels

$$y \in \{1, \ldots, K\},$$

where n is the number of examples, d is the number of features, and K is the number of classes.

The learning objective is to estimate a function that predicts class membership through probabilities:

$$f(x) = \arg\max_k p(y = k \mid x).$$

This is a probabilistic formulation of classification. Rather than directly outputting only a class label, the model first assigns a probability to each class, and the final prediction is the class with the largest probability. This is useful because it gives not only a decision, but also a measure of confidence.

Softmax Regression as a Probabilistic Model

Softmax regression is a linear classifier. For an input x, it computes class scores

$$s = Wx + b,$$

or in matrix form for a batch,

$$S = XW^T + b.$$

These raw scores are transformed into probabilities by the softmax function:

$$p(y = k \mid x) = \frac{e^{s_k}}{\sum_j e^{s_j}}.$$

Because the softmax converts arbitrary real-valued scores into non-negative values that sum to one, the model defines a valid probability distribution over classes. That is why softmax regression is not merely a scoring rule, but a probabilistic classification model.

Although the output is probabilistic, the dependence on the input remains linear through the logits. Therefore, the resulting classifier can only represent linear decision boundaries in the original feature space.

Cross-Entropy as Negative Log-Likelihood

For a training dataset, the model parameters are learned by minimizing cross-entropy loss:

$$L = -\frac{1}{n} \sum_{i=1}^{n} \log p(y_i \mid x_i).$$

This expression is the negative log-likelihood of the observed labels under the probabilistic model. If the true class receives high predicted probability, then the log-probability is close to zero and the loss is small. If the true class receives low probability, the loss becomes large. Thus, minimizing cross-entropy is exactly equivalent to maximizing the likelihood of the observed data under the model.

To reduce overfitting, L_2 regularization is added:

$$L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2}\|W\|_2^2$$

for softmax regression, and analogously for neural-network weight matrices. This penalizes overly large parameter values and encourages smoother solutions.

One-Hidden-Layer Neural Network

The neural network used in this project has one hidden layer. Structurally, it performs:

$$Z_1 = XW_1^T + b_1,$$

$$H = \tanh(Z_1),$$

$$S = HW_2^T + b_2.$$

The first affine transformation maps inputs into a hidden representation. The tanh activation is then applied elementwise, introducing non-linearity. The final output layer maps the hidden representation to class logits, which are then passed through softmax to obtain class probabilities.

This model is fundamentally different from softmax regression because it does not classify directly in the original space. Instead, it first transforms the data into a new learned feature space. Without the non-linear activation, two linear layers would collapse into a single linear transformation. Therefore, the hidden non-linearity is exactly what gives the model additional expressive power.

Datasets and Geometric Motivation

The project uses three datasets:

- **Linear Gaussian dataset**: two approximately Gaussian clusters that are linearly separable
- **Moons dataset**: a non-linear two-class dataset with curved class structure
- **Digits dataset**: an 8×8 handwritten digit image benchmark flattened into 64-dimensional vectors with 10 classes

These datasets differ in their geometry. The Gaussian dataset should be solvable by a straight boundary, so a linear model is expected to be sufficient. The moons dataset cannot be correctly separated by any straight line, so a representationally richer model is needed. The digits dataset is more realistic, with high dimensionality, multiple classes, and greater variation.

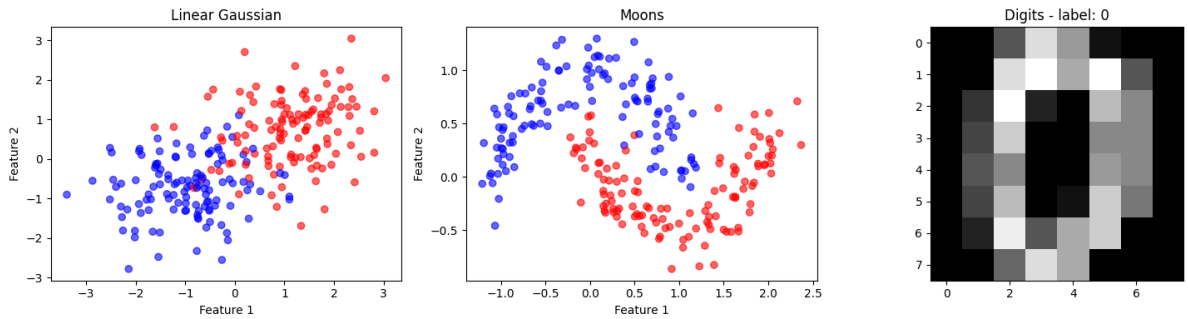

Figure 1: Overview of the three datasets used in the project: Linear Gaussian, Moons, and the Digits benchmark. The synthetic datasets illustrate low-dimensional geometric structure, while the digits example highlights the image-based multi-class setting.

Figure 1 provides a visual summary of the data geometry. The Gaussian and moons datasets make the contrast between linear and non-linear structure immediately visible, while the digits example shows the more realistic multi-class image-based setting used later in the report.

3 Methods

Data Loading and Preprocessing

All datasets were loaded through NumPy-based utilities. The data was already preprocessed and normalized to the range $[0, 1]$, so no additional feature scaling or transformation was applied. No external datasets were added, and no extra feature engineering was introduced. This keeps the focus of the project on model behavior rather than preprocessing tricks.

For the Gaussian and moons datasets, separate training, validation, and test arrays were directly loaded from the provided files. For the digits benchmark, the full feature matrix and labels were loaded first, and then training, validation, and test subsets were selected using the provided index arrays. This guarantees reproducibility and ensures compliance with the capstone specification.

Models

Softmax Regression. The softmax regression model was implemented fully from scratch with:

- numerically stable softmax
- vectorized gradient computation
- cross-entropy loss with L_2 regularization
- mini-batch stochastic gradient descent

A stable softmax implementation is important because exponentials may overflow for large logits. To avoid this, the maximum value in each row is subtracted before exponentiation. This preserves the probabilities while improving numerical stability.

One-Hidden-Layer Neural Network. The neural network uses one hidden layer with tanh activation:

$$Z_1 = XW_1^T + b_1, \qquad H = \tanh(Z_1), \qquad S = HW_2^T + b_2.$$

The final probabilities are produced by softmax. The model is trained with backpropagation, where gradients are computed analytically and fully vectorized.

For the output layer, the derivative of the loss with respect to the logits is:

$$dS = \frac{1}{n}(P - Y),$$

where P is the predicted probability matrix and Y is the one-hot label matrix.

The second-layer gradients are:

$$dW_2 = dS^T H + \lambda W_2, \qquad db_2 = \sum_i dS_i.$$

Backpropagation into the hidden layer gives:

$$dH = dSW_2.$$

Since

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z),$$

we obtain

$$dZ_1 = dH \odot (1 - H^2).$$

Then the first-layer gradients are:

$$dW_1 = dZ_1^T X + \lambda W_1, \qquad db_1 = \sum_i dZ_{1,i}.$$

Training Protocol

Both models were trained using mini-batch optimization with:

- batch size 64
- learning rates in the range 0.05 to 0.1
- up to 200 epochs
- L_2 regularization
- validation-based model selection

Mini-batch optimization provides a balance between full-batch and purely stochastic updates. It makes gradient estimates reasonably stable while remaining computationally efficient.

Validation-based model selection was critical. Rather than automatically using the last epoch, the best model checkpoint was selected based on validation performance. This reduces the chance of overtraining and supports a fairer comparison.

Optimizer Choices

The main training setup used mini-batch SGD. In an additional optimizer study on the digits benchmark, the neural network was also trained with Momentum and Adam in order to compare convergence speed and validation behavior.

Evaluation Metrics

The evaluation considered:

- classification accuracy
- cross-entropy loss
- training and validation dynamics across epochs
- decision-boundary quality on low-dimensional datasets
- confidence and uncertainty structure on digits

Beyond accuracy alone, the project examined predictive confidence, entropy, and probability calibration, since a good classifier should ideally be both accurate and reliable.

Digits Protocol

The digits dataset required especially careful protocol handling because it is the main realistic benchmark in the project. The split into training, validation, and test sets was predefined and kept fixed throughout all experiments.

The usage of each subset was explicit:

- **Training set**: used to update model parameters
- **Validation set**: used for hyperparameter comparison and best-checkpoint selection
- **Test set**: used only for final reporting and comparison

This means that the test set was not used for model tuning, early stopping decisions, or optimizer selection. That separation is important because it keeps the final test metrics meaningful as estimates of generalization.

4 Implementation Sanity Checks

Several verification checks were used to confirm that the implementations were correct.

First, a **probability-sum check** verified that model outputs formed valid distributions. For each input, the predicted class probabilities were checked to ensure they summed to one.

Second, a **gradient sanity check** compared analytic gradients to numerical finite-difference approximations. This is especially important in from-scratch implementations, because a small algebraic mistake in backpropagation can produce misleading results even if training appears to proceed.

Third, a **tiny-subset loss decrease test** was used to confirm that the optimizer could successfully reduce the training loss on a very small dataset. This ensured that the update rules and learning-rate choices were functioning sensibly.

Fourth, a **tiny-subset overfitting test** checked whether the model could fit a very small batch almost perfectly. If a sufficiently expressive model cannot memorize a tiny subset, that often indicates an implementation bug or optimization failure.

Finally, **NaN/Inf checks** were used during training to confirm numerical stability. This was especially important because softmax and exponential computations can become unstable if implemented carelessly.

Together, these checks provided evidence that the code was functioning correctly and that the experimental conclusions were based on valid implementations rather than accidental behavior.

5 Experiments

The experiments were designed to answer a set of interpretive questions: when a linear model is enough, what representational advantage the hidden layer provides, when extra complexity is not justified, what failure cases reveal, and what repeated-seed or repeated-run style evidence adds beyond a single example.

Core Comparison Across Datasets

Linear Gaussian Dataset. On the Gaussian dataset, both softmax regression and the neural network achieved near-perfect performance. This result is important because it shows that a linear model is already sufficient when the data geometry is linearly separable. In this setting, the neural network does not deliver a meaningful practical gain. That is not a weakness of the neural network, but rather evidence that added complexity is unnecessary for simple problems.

Moons Dataset. On the moons dataset, the difference between the models was clear. Softmax regression performed poorly because it is restricted to a straight boundary, while the neural network learned a curved separator and achieved much stronger performance. This demonstrates that the issue is not merely optimization, but representation. A linear classifier cannot solve a problem whose class geometry is inherently non-linear.

Digits Dataset. On the digits benchmark, softmax regression served as a strong baseline with moderate accuracy, but the neural network achieved better performance overall. The neural network also showed improved validation loss and stronger confidence structure in predictions. This suggests that increased model complexity is useful once the dataset contains high-dimensional structure and more subtle class interactions.

Decision-Boundary Comparison

A central experiment involved visualizing the learned decision boundaries on the Gaussian and moons datasets.

On the Gaussian dataset, both models learned nearly identical boundaries and achieved comparable accuracy. This is direct geometric evidence that the linear model is sufficient there.

On the moons dataset, softmax regression was visibly constrained to a straight boundary that cut through both classes incorrectly. The neural network, in contrast, learned a curved separator that followed the moon-shaped structure much more effectively. This is perhaps the clearest demonstration in the project of what the hidden layer provides: not merely more parameters, but a new representational geometry.

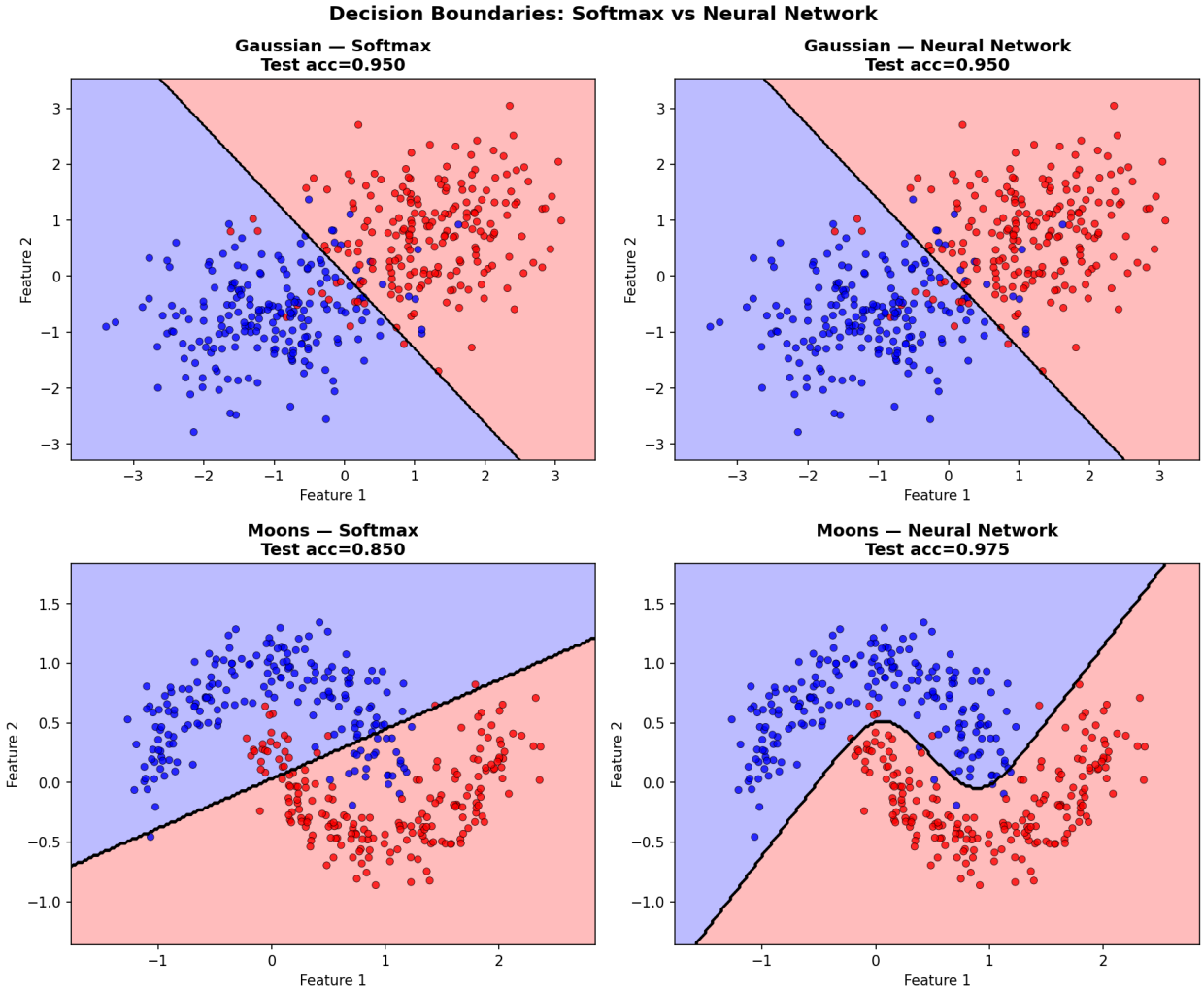

Figure 2: Decision boundary comparison between softmax regression and the one-hidden-layer neural network on the Gaussian and moons datasets. On Gaussian, both models learn nearly identical linear boundaries and achieve the same test accuracy. On moons, softmax is constrained to a straight boundary, while the neural network learns a curved separator and achieves much higher test accuracy.

Qualitative Model Comparison

The overall qualitative comparison can be summarized as follows:

Dataset	Softmax Regression	Neural Network
Gaussian	Excellent	Excellent
Moons	Poor	Excellent
Digits	Moderate	Better

The key point is not simply that the neural network is better in general. Rather, its advantage depends strongly on the structure of the data.

Capacity Ablation

A hidden-capacity ablation study was performed on the moons dataset using hidden dimensions such as $\{2, 8, 32\}$.

With very small hidden width, the model underfit and still behaved too much like a nearly linear separator. With moderate width, the representation improved somewhat but remained limited. With a larger hidden width, the network could trace the curved class structure much more effectively and produced substantially better accuracy.

This experiment shows that using a neural network is not enough by itself. The hidden layer must also have enough capacity to represent the target geometry. Thus, extra complexity only becomes useful once it is sufficient to express the needed non-linearity.

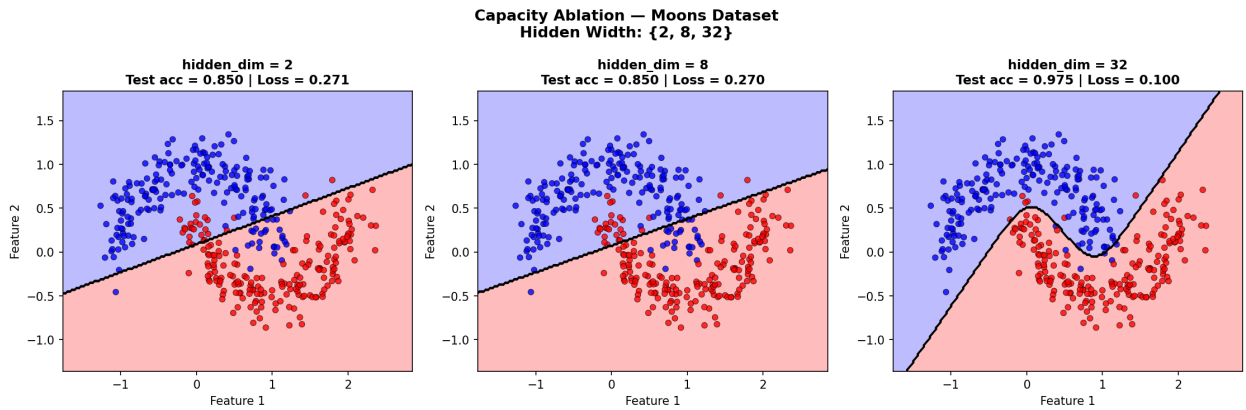

Figure 3: Capacity ablation study on the moons dataset for hidden dimensions $\{2, 8, 32\}$. Small and medium hidden widths still behave close to a nearly linear separator and achieve limited performance, while the larger hidden width learns a much more flexible non-linear boundary and substantially improves results.

Training Dynamics on Digits

Training and validation curves on the digits dataset revealed another important pattern. The softmax model started from a stronger initial position because it is simpler and easier to optimize. The neural network initially had higher loss and lower accuracy. However, over time, the neural network improved rapidly and eventually surpassed softmax regression.

This means that early optimization difficulty should not automatically be interpreted as a worse final model. More expressive models may need more time to organize a useful hidden representation, but that added effort can produce better final performance.

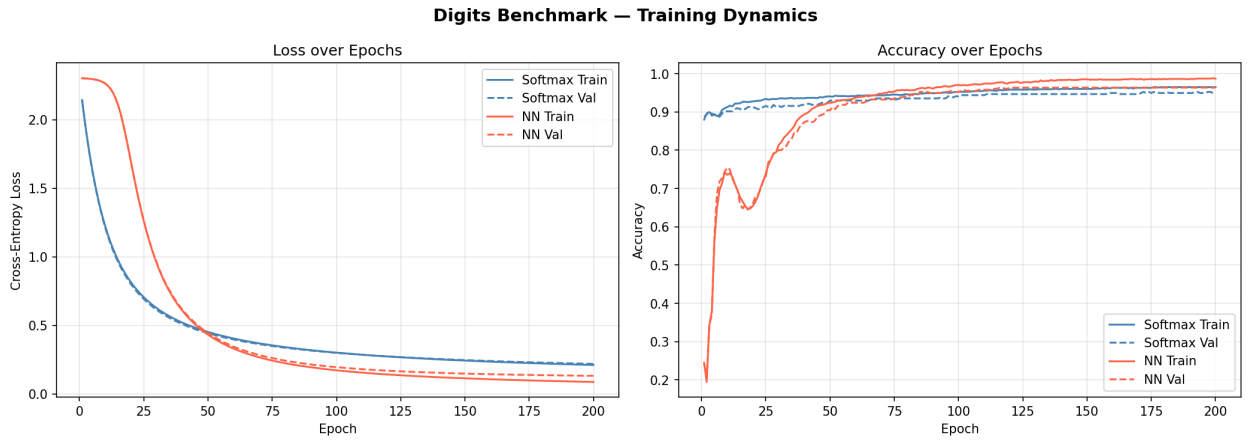

Figure 4: Training and validation dynamics on the digits benchmark. The neural network starts with higher loss and lower accuracy but improves rapidly and eventually surpasses the softmax model in both cross-entropy and accuracy.

Optimizer Study

An optimizer study on the digits benchmark compared plain SGD, Momentum, and Adam for the neural network. Standard SGD converged steadily but more slowly. Momentum improved convergence speed by carrying directional information across steps. Adam also converged rapidly by adapting learning rates for different parameters.

The study showed that optimizer choice can strongly affect training efficiency even when architecture is fixed. While all methods were capable of learning, Momentum and Adam reached strong validation performance earlier than plain SGD. This does not change the main representational conclusions of the project, but it does show that training behavior depends not only on the model class, but also on the optimization method.

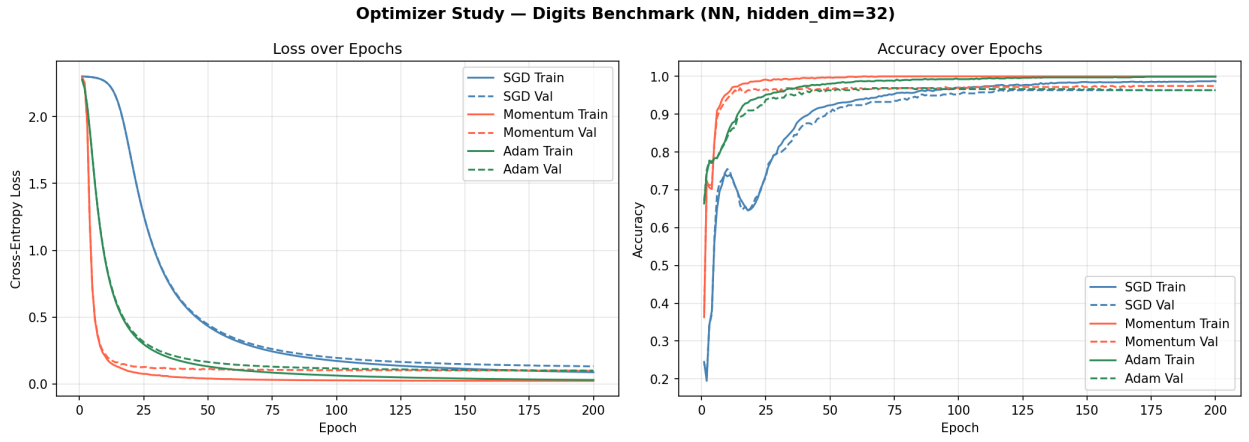

Figure 5: Optimizer comparison on the digits benchmark for the neural network. Momentum and Adam converge faster than plain SGD and reach strong validation performance earlier in training.

Failure Case and What It Teaches

The clearest failure case in the project is the softmax model on the moons dataset. The important lesson is that poor results do not always imply poor training. In this case, even a correct implementation and successful optimization cannot compensate for a representational mismatch. The model fails because it is asked to express a curved boundary using a linear rule. This distinction between *optimization failure* and *representation failure* is one of the central conceptual lessons of the capstone.

Repeated-Seed Perspective

Even though the report focuses primarily on interpretable core experiments rather than exhaustive statistical sweeps, the project framework highlights why repeated-seed or repeated-run analysis matters. A single run may reflect random initialization or mini-batch noise. Repeated-seed summaries can show whether an observed advantage is robust or merely accidental. In that sense, repeated-run evidence adds another layer of confidence beyond a single plot or one final metric, especially for neural networks where initialization can affect early training behavior.

6 Advanced Track

Track B: Prediction confidence and reliability

This project includes the confidence and reliability track on the digits benchmark.

Confidence Histogram

A confidence histogram compared the maximum predicted probability for correct and incorrect predictions. In both models, correct predictions tended to have higher confidence than incorrect ones. However, the neural network showed a stronger concentration of highly confident correct predictions. This suggests that its improved accuracy is accompanied by more decisive classification behavior.

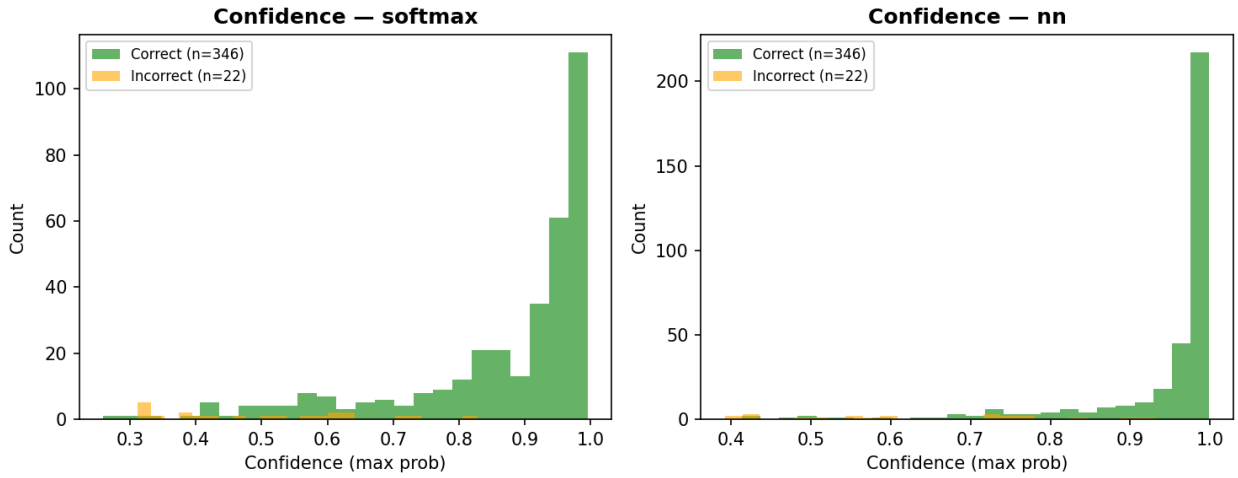

Figure 6: Histogram of predictive confidence for correct and incorrect predictions on the digits benchmark. The neural network produces a stronger concentration of highly confident correct predictions and a clearer separation between correct and incorrect cases.

Entropy Analysis

Predictive entropy provided another view of uncertainty. Lower entropy indicates that the model concentrates most probability mass on one class, while higher entropy reflects uncertainty spread across several classes.

The entropy analysis showed that correct predictions generally had lower entropy than incorrect predictions for both models. The neural network had especially low entropy for many correct predictions, indicating stronger certainty when it was right.

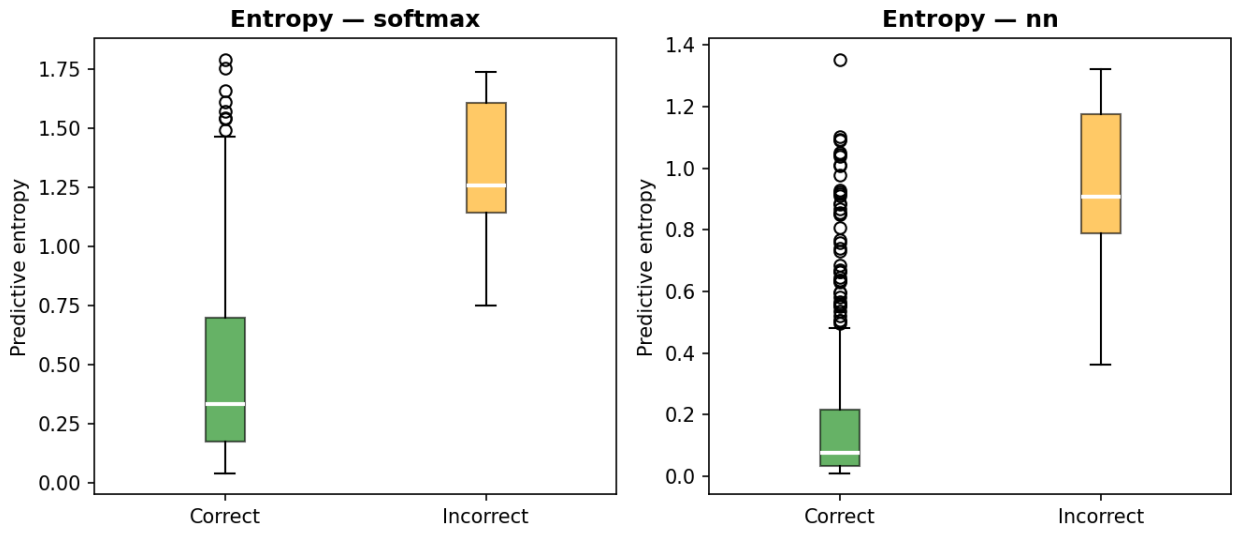

Figure 7: Predictive entropy for correct and incorrect predictions. Correct predictions tend to have lower entropy, while incorrect predictions have higher entropy. The neural network shows especially low entropy for many correct predictions, indicating stronger certainty.

Reliability Analysis

Reliability diagrams compared predicted confidence against empirical accuracy. A perfectly calibrated model would lie near the diagonal, meaning that, for example, predictions made at 80% confidence would be correct about 80% of the time.

The reliability results suggested that the neural network was closer to the ideal line, especially in higher-confidence regions. This indicates that its probabilities were not only useful for ranking predictions, but also somewhat more trustworthy as estimates of correctness.

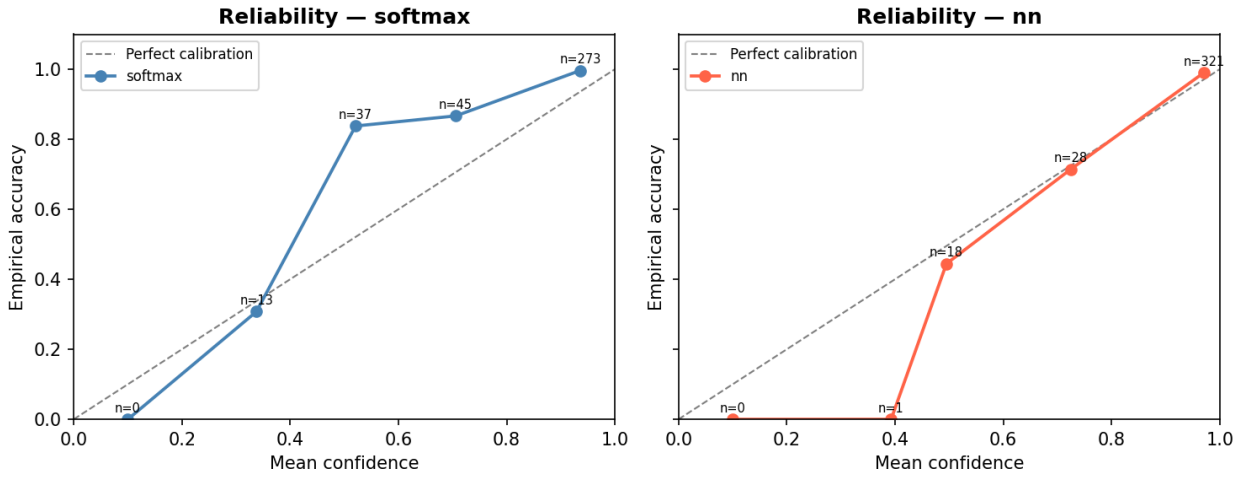

Figure 8: Reliability diagrams for softmax regression and the neural network. The dashed diagonal indicates perfect calibration. The neural network lies closer to the ideal line across important confidence regions, suggesting better calibration.

Taken together, these analyses show that model evaluation should go beyond raw accuracy. A model that is both accurate and well-calibrated is usually more useful than one that is only accurate.

7 Discussion

The results support a consistent interpretation across all experiments.

First, **linear structure is sufficient when the data itself is approximately linearly separable**. This is clearly demonstrated by the Gaussian dataset, where softmax regression and the neural network perform almost identically.

Second, **non-linearity helps when the geometry of the task requires it**. The moons dataset is the strongest example. There, the hidden layer makes a real difference because it enables the model to transform the feature space and learn a curved boundary that a linear classifier cannot represent.

Third, the **digits benchmark lies between these two extremes**. It is not a toy non-linear example in the same visual sense as moons, but it does contain enough complexity that the neural network's hidden representation leads to better final performance. The digits results therefore show that the value of non-linearity extends beyond obvious curved toy datasets into more realistic settings.

The experiments also make clear that **extra complexity does not always justify itself**. On the Gaussian dataset, the neural network does not yield meaningful gains. This is important because it shows that "more powerful" does not automatically mean "more appropriate." Model complexity should be matched to data structure.

Similarly, the project distinguishes carefully between what is supported by evidence and what remains uncertain. The evidence strongly supports the claim that the hidden layer is useful on non-linear and higher-structure tasks. The evidence also supports the claim that linear models remain strong baselines and may be entirely sufficient in simple settings. However, the project does not claim that one hidden layer is universally optimal, nor that the observed results would remain unchanged under all datasets, all hyperparameters, or all architectures.

In summary, the capstone shows that good model selection depends on understanding geometry, representational capacity, optimization behavior, and uncertainty together rather than treating final accuracy as the only criterion.

8 Limitations

This project has several limitations that define the scope of its conclusions.

First, only one hidden-layer neural network was studied. While this is sufficient for isolating the role of non-linearity, it does not test whether deeper architectures would produce qualitatively different conclusions on the digits benchmark.

Second, hyperparameter exploration was intentionally limited. Learning rates, hidden sizes, regularization, and optimizer settings were explored enough to support the report's main claims, but not through an exhaustive search.

Third, the dataset set was deliberately small. The Gaussian and moons datasets are useful because they make geometric interpretation easy, and the digits dataset adds a realistic benchmark, but broader generalization across many domains was not tested.

Fourth, advanced regularization strategies and broader robustness studies were not included. Similarly, repeated-seed statistics were discussed conceptually but not extended into a fully exhaustive statistical study across all experiments.

Therefore, the conclusions should not be overgeneralized. The project supports claims about the relationship between data geometry and model class within the controlled settings studied here. It does not claim that shallow neural networks are always preferable to linear models, nor that these exact patterns would automatically transfer to every real-world dataset.

References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning.* MIT Press.

2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning.* Springer.

3. Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective.* MIT Press.

4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.

5. Nielsen, M. A. (2015). *Neural Networks and Deep Learning.* Determination Press.

6. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

7. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)* (pp. 1321–1330).

8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.